

Efficient PE Header Feature Extraction for Malware Analysis: Comparing Reduced and Extended Feature Sets

Anonymous Authors

Affiliation withheld for double-blind review

Abstract—Malware detection remains a critical challenge in cybersecurity as the volume and sophistication of malicious software continue to increase exponentially. Notably, prior studies on the APT1 dataset reported feature extraction times exceeding 19 minutes (1,157.81 seconds) for processing executable samples, making real-time detection impractical. This research presents an efficient feature extraction methodology for static malware analysis utilizing Portable Executable (PE) header files. We conduct a comprehensive comparative study between two distinct feature set approaches: a reduced feature set (14 features) and an extended feature set (32 features) extracted from PE file headers. Our experiments, conducted on a newly created and publicly available dataset of 1,150 malware executable samples collected from MalwareBazaar, demonstrate that the extended feature set achieves superior classification accuracy (94.78%) compared to the reduced feature set (93.91%) using Random Forest classification, with Decision Tree also achieving 93.48% accuracy on the extended set. Our proposed methodology reduces feature extraction time from approximately 19 minutes to approximately 215–290 seconds, representing a reduction of over 75%. The research evaluates six machine learning classifiers including Random Forest, K-Nearest Neighbors, Decision Tree, Support Vector Machine, Logistic Regression, and Gradient Boosting, providing insights into the trade-offs between feature dimensionality, extraction time, and classification performance. Additionally, we create and publish a novel malware dataset with pre-extracted PE header features for research purposes. Our findings contribute to the development of more efficient real-time malware detection systems by reducing feature extraction time.

Index Terms—PE Header, Feature Extraction, Extraction Time, Machine Learning, Cybersecurity

I. INTRODUCTION

The proliferation of malware poses a critical threat to computer systems worldwide, with over 1 billion instances detected in 2023 alone, approximately 33 new samples per second [1]. Detection pipelines must triage this volume near real-time, making the cost of feature extraction a practical concern alongside detection accuracy. Static malware analysis examines executable files without execution [5], offering safety and speed advantages over dynamic sandboxed approaches [2], and the ability to inspect samples that employ runtime anti-analysis. Among static signals, the Portable Executable (PE) format [4] is attractive because its `FILE_HEADER` and `OPTIONAL_HEADER` expose dozens of metadata fields (machine type, entry point, image base, section/file alignment, subsystem, DLL characteristics, stack/heap sizes) [6] whose

distributions differ between malicious and benign binaries [7], [9], making them cheap features for static classification.

Prior PE-header studies on the APT1 dataset [3] reported feature extraction times exceeding 19 minutes (1,157.81 s) per corpus, impractical for detection systems that process thousands of samples daily, especially given that ransomware attacks occurred every ~ 11 seconds in 2021. Comprehensive feature sets capture richer signal at the cost of higher extraction time; minimal sets sacrifice discriminative power for speed. Prior work has not systematically compared feature-set sizes against extraction time on a single modern dataset.

Contributions. This paper develops an efficient PE-header feature extraction methodology and reports a measurement study of the feature-set-size trade-off. We compare a reduced 14-attribute set (PE-header fields most commonly used in prior work) against an extended 32-attribute set (all accessible `FILE_HEADER` and `OPTIONAL_HEADER` fields), evaluate six classifiers (Random Forest, Decision Tree, Gradient Boosting, K-Nearest Neighbors, Logistic Regression, Support Vector Machine), and report extraction throughput on a newly created 1,150-sample MalwareBazaar corpus released with the paper. Our pipeline reduces extraction time from ~ 19 minutes (prior APT1 reports) to 215–290 seconds, a $>75\%$ reduction; the extended set reaches 94.78% accuracy with Random Forest while the reduced set reaches 93.91%, and KNN improves by 4.78 pp moving from reduced to extended.

II. WORKING MECHANISM

A. System Architecture

The proposed malware detection system comprises three primary phases: (1) Feature Extraction from PE files using the `pefile` library, (2) Machine Learning Classification using trained models, and (3) Detection Result output. Figure 1 illustrates this workflow.

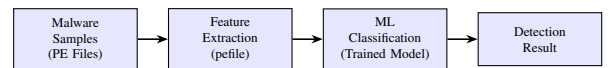


Fig. 1: System Architecture Overview

B. Dataset Preparation

1) *Data Collection:* Malware executable samples were collected from MalwareBazaar [11], a reputable malware sample

repository. The dataset comprises 1,150 PE executable files representing various malware families.

2) *Data Processing*: The Python library was employed for PE header parsing. Files were validated to ensure proper PE format before feature extraction. Non-PE files (.rar, .elt, etc.) were excluded from the dataset.

C. Feature Extraction Methodology

The PE bitfield is the classification target. The **reduced (14-feature) set** draws , from FILE_HEADER and 12 OPTIONAL_HEADER fields (, , , , , OS/image major+minor versions, ,). The **extended (32-feature) set** adds 4 FILE_HEADER fields (, , ,) and 14 OPTIONAL_HEADER fields covering linker versions, code/data sizes, base of code, subsystem versions, , stack/heap reserve+commit, , and .

D. Feature Extraction Procedure

The feature extraction process is implemented using Python with the library. Algorithm 1 presents the pseudocode for the extraction process.

Algorithm 1 PE Header Feature Extraction

Require: Directory path containing PE files

Ensure: CSV file with extracted features

```

1: for each file in directory do
2:   Parse PE file using pefile library
3:   if PE parsing successful then
4:     Extract FILE_HEADER features
5:     Extract OPTIONAL_HEADER features
6:     Append features to dataset
7:   else
8:     Log error and skip file
9:   end if
10: end for
11: Save dataset to CSV file

```

E. Machine Learning Classification

Six machine learning classifiers were employed for malware detection. Random Forest is an ensemble method using 100 decision trees with bootstrap aggregation. K-Nearest Neighbors (KNN) is an instance-based learning approach using Euclidean distance with $k = 5$. Decision Tree performs tree-based classification using information gain. Support Vector Machine (SVM) uses kernel-based classification with an RBF kernel. Logistic Regression performs linear classification with maximum likelihood estimation. Gradient Boosting is a sequential ensemble method with 50 estimators.

F. Experimental Setup

The dataset was split into 80% training (920 samples) and 20% testing (230 samples). Model validation was performed using 3-fold cross-validation, consistent across both feature sets. A random state of 42 was used for reproducibility. The evaluation metrics employed include Accuracy, Precision, Recall, and F1-Score.

III. RESULTS

A. Feature Extraction Performance

Table I presents the feature extraction performance comparison between the two approaches.

TABLE I: Feature Extraction Performance Comparison

Metric	Reduced (14)	Extended (32)
Extraction Time	215.81 sec	290.12 sec
Processing Speed	5.33 files/sec	3.96 files/sec
Files Processed	1,150	1,150
Errors	0	0
CSV Columns	16	34

Comparison with Prior Work (APT1 Dataset):

- APT1 PE Header Extraction: 1,157.81 seconds (~19 minutes)
- Our Extended Feature Set: 290.12 seconds (~4.8 minutes)
- **Improvement: 75% reduction in extraction time**

B. Classification Results - Reduced Feature Set

Table II presents the classification results for the reduced feature set (14 features).

TABLE II: Classification Results - Reduced Feature Set (14 Features)

Classifier	Accuracy	F1	Precision	Recall
Random Forest	93.91%	0.8379	0.8858	0.8264
Decision Tree	91.74%	0.7329	0.7681	0.7302
Gradient Boosting	90.87%	0.5956	0.6037	0.6028
KNN	75.22%	0.4636	0.4969	0.4590
Logistic Regression	51.30%	0.1369	0.1309	0.1734
SVM	25.65%	0.0240	0.0151	0.0588

As shown in Table II, tree-based classifiers dominate performance with the reduced feature set. Random Forest achieves the highest accuracy of 93.91% with an F1-score of 0.8379 and the best precision of 0.8858, indicating that ensemble methods effectively leverage even a limited set of PE header features for malware classification. Decision Tree follows closely at 91.74% accuracy with an F1-score of 0.7329, while Gradient Boosting achieves 90.87% accuracy but with a notably lower F1-score of 0.5956, suggesting it struggles with certain malware classes despite reasonable overall accuracy. KNN shows moderate performance at 75.22%, indicating that distance-based methods have difficulty distinguishing malware families in the limited 14-dimensional feature space. Logistic Regression and SVM perform poorly at 51.30% and 25.65% respectively, which is likely attributable to the multi-class nature of the classification problem and the absence of feature scaling, both of which are critical for optimal performance of these linear classifiers.

C. Classification Results - Extended Feature Set

Table III presents the classification results for the extended feature set (32 features).

TABLE III: Classification Results - Extended Feature Set (32 Features)

Classifier	Accuracy	F1	Precision	Recall
Random Forest	94.78%	0.8632	0.9051	0.8399
Decision Tree	93.48%	0.7993	0.8167	0.7927
Gradient Boosting	93.04%	0.7209	0.7616	0.7133
KNN	80.00%	0.5085	0.5300	0.5053
Logistic Regression	41.30%	0.1098	0.1076	0.1368
SVM	25.65%	0.0240	0.0151	0.0588

With the extended feature set of 32 attributes, all tree-based classifiers show notable improvement over the reduced set. Random Forest achieves the best accuracy of 94.78% with an F1-score of 0.8632 and precision of 0.9051, confirming that additional PE header features provide more discriminative power for ensemble methods. Decision Tree improves to 93.48% accuracy with an F1-score of 0.7993, and Gradient Boosting reaches 93.04% with a substantially improved F1-score of 0.7209 compared to 0.5956 in the reduced set, indicating that the additional features help resolve previously ambiguous classifications. KNN benefits the most from the extended features, rising from 75.22% to 80.00%, a 4.78 percentage point improvement, which suggests that the additional features create more meaningful distance separations in the higher-dimensional feature space. Notably, Logistic Regression drops from 51.30% to 41.30%, likely due to multicollinearity among the additional correlated features, which degrades the performance of linear models. SVM remains unchanged at 25.65%, confirming its unsuitability for this multi-class problem without proper feature scaling and kernel optimization.

D. Comparative Analysis

Table IV presents the accuracy improvement achieved by the extended feature set across all six classifiers.

TABLE IV: Accuracy Comparison Between Feature Sets

Classifier	Reduced	Extended	Improvement
Random Forest	93.91%	94.78%	+0.87%
Decision Tree	91.74%	93.48%	+1.74%
Gradient Boosting	90.87%	93.04%	+2.17%
KNN	75.22%	80.00%	+4.78%
Logistic Regression	51.30%	41.30%	-10.00%
SVM	25.65%	25.65%	0.00%

Table IV reveals a clear pattern in how different classifier families respond to additional features. Tree-based and instance-based classifiers consistently benefit from the extended feature set, with improvements ranging from +0.87% for Random Forest to +4.78% for KNN. The relatively small improvement for Random Forest (+0.87%) suggests that it already captures most of the discriminative information with just 14 features, while KNN's substantial improvement (+4.78%) indicates that the additional features create more separable clusters in higher-dimensional space, enabling better distance-based classification. Gradient Boosting shows a +2.17% improvement, demonstrating that sequential ensemble methods

also benefit from richer feature representations. In contrast, Logistic Regression suffers a 10% drop in accuracy with the extended set, highlighting the curse of dimensionality for linear models when features are correlated. SVM shows no change, remaining at 25.65% for both sets, which confirms that without proper feature scaling and hyperparameter optimization, kernel-based methods cannot effectively leverage PE header features for multi-class malware classification. These results suggest that the choice of feature set should be guided by the intended classifier: tree-based methods can work well with either set, while linear classifiers require careful feature engineering.

E. Feature Importance Analysis

Table V presents the top 10 most important features for malware classification using Random Forest on the extended feature set.

TABLE V: Top 10 Feature Importance (Random Forest)

Rank	Feature	Importance
1	AddressOfEntryPoint	0.095
2	MajorLinkerVersion	0.092
3	TimeStamp	0.084
4	SizeOfCode	0.068
5	DllCharacteristics	0.065
6	ImageBase	0.057
7	SizeOfInitializedData	0.057
8	SizeOfOptionalHeader	0.050
9	Machine	0.050
10	NumberOfSections	0.039

F. Performance Visualization - Reduced Feature Set

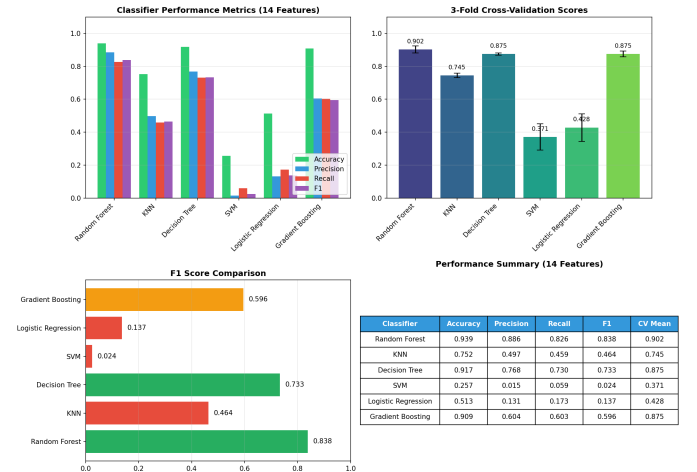


Fig. 2: Classifier Performance Comparison - Reduced Feature Set (14 Features)

Random Forest dominates all four metrics; a sharp drop-off from Gradient Boosting to KNN marks the boundary between tree-based and non-tree-based methods.

Tree-based methods (RF 93.91%, DT 91.74%, GB 90.87%) cluster at the top; KNN (75.22%) forms a middle tier; LR and

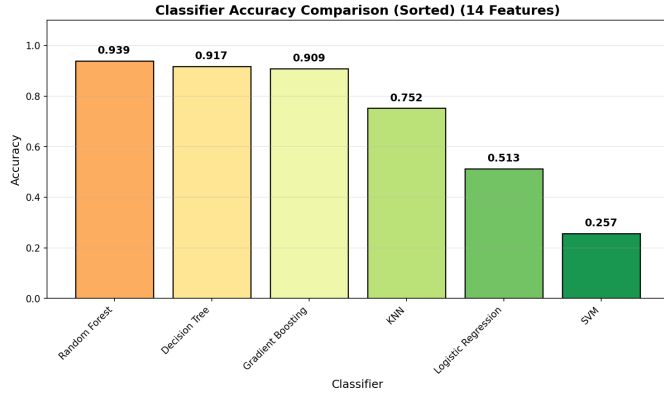


Fig. 3: Accuracy Comparison - Reduced Feature Set (14 Features)

SVM perform near or below random chance for this multi-class task.

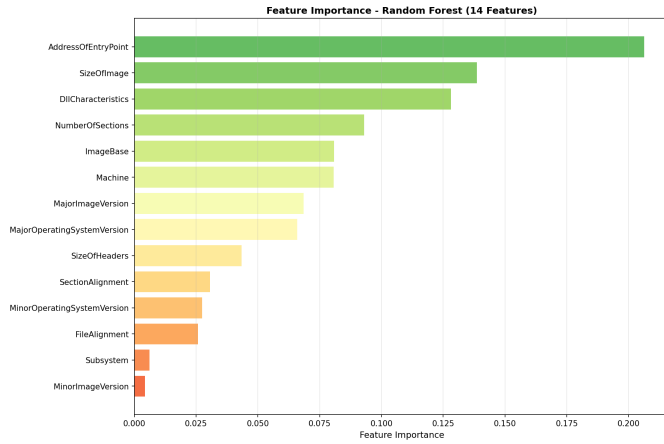


Fig. 4: Feature Importance - Reduced Feature Set (14 Features)

, , and dominate; a few features contribute disproportionately, suggesting further reduction is possible without major accuracy loss.

Random Forest and Decision Tree show strong diagonal dominance; SVM and Logistic Regression exhibit scattered predictions, confirming their inability to distinguish classes with this feature set.

Random Forest covers the largest area, indicating balanced superiority; SVM and LR shapes collapse near the center.

G. Performance Visualization - Extended Feature Set

Figure 7 shows the classifier performance comparison for the extended feature set.

Bars for RF, DT, GB, and KNN are taller than in the reduced set across all metrics; the GB-to-KNN gap narrows, suggesting extended features help instance-based methods more proportionally.

RF leads at 94.78%, followed by DT (93.48%) and GB (93.04%); KNN improves notably to 80.00%; LR and SVM

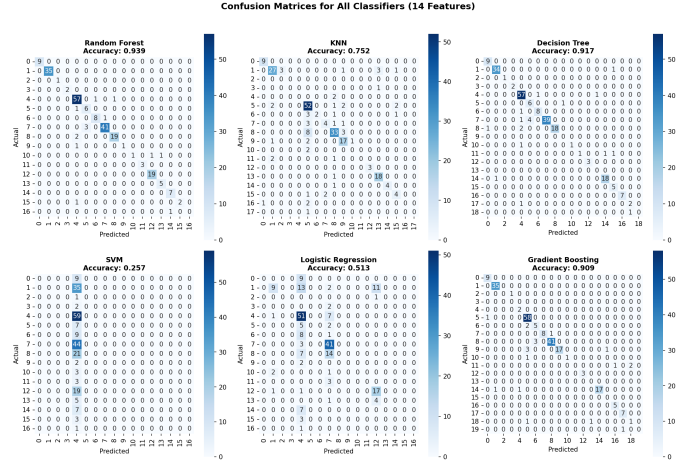


Fig. 5: Confusion Matrices - Reduced Feature Set (14 Features)

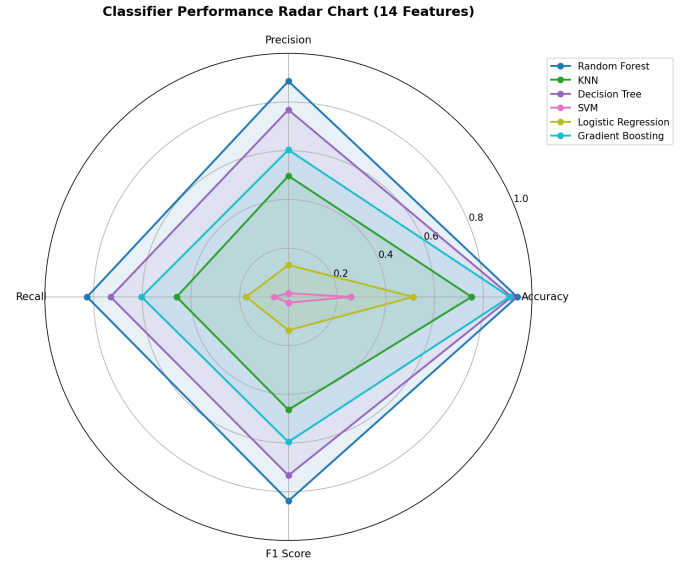


Fig. 6: Performance Radar Chart - Reduced Feature Set (14 Features)

remain poor, showing that additional features do not compensate for fundamental classifier limitations.

The importance distribution is more gradual than in the reduced set, with , , and leading; a long tail of small-but-nonzero importances accounts for the observed accuracy improvement.

Diagonal entries are more prominent than in Figure 5 for RF, DT, GB, and KNN; SVM and LR remain largely unchanged.

RF's coverage area is the largest and most symmetric; KNN's shape expands relative to Figure 6, while SVM and LR remain near the center.

IV. CONCLUSION AND FUTURE WORK

We presented a comparative study of PE header-based feature extraction for static malware analysis. Our pipeline

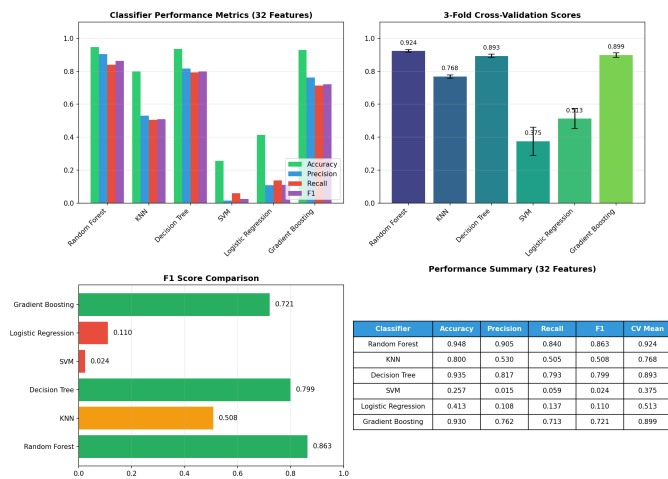


Fig. 7: Classifier Performance Comparison - Extended Feature Set (32 Features)

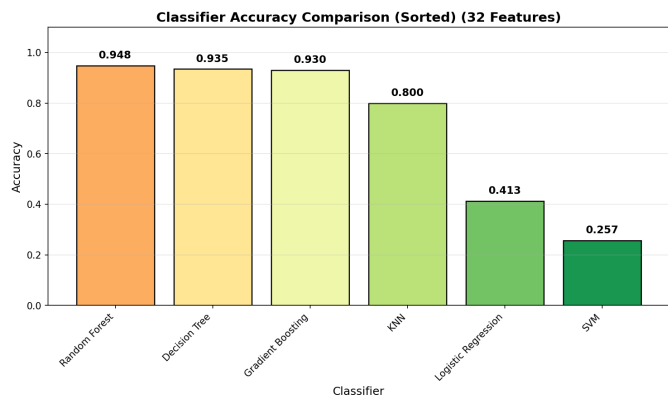


Fig. 8: Accuracy Comparison - Extended Feature Set (32 Features)

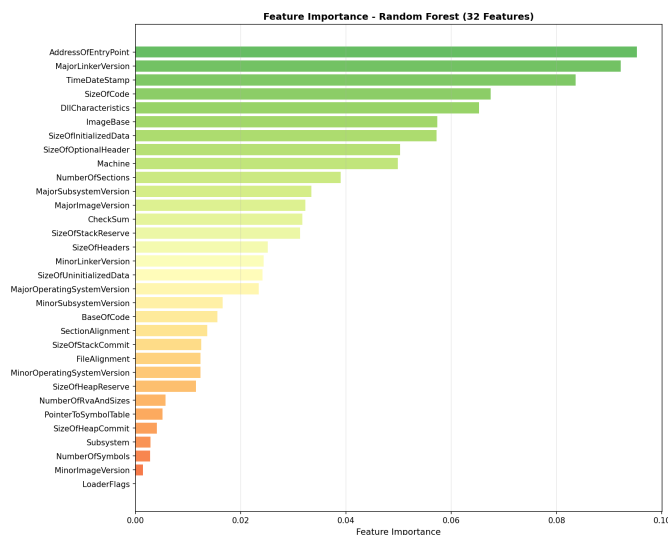


Fig. 9: Feature Importance - Extended Feature Set (32 Features)

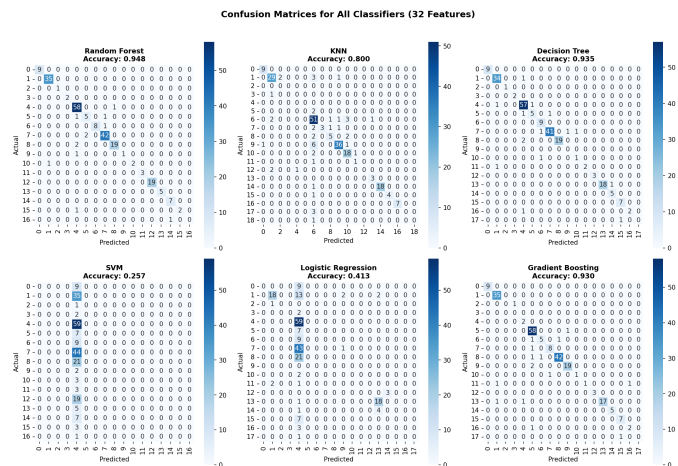


Fig. 10: Confusion Matrices - Extended Feature Set (32 Features)

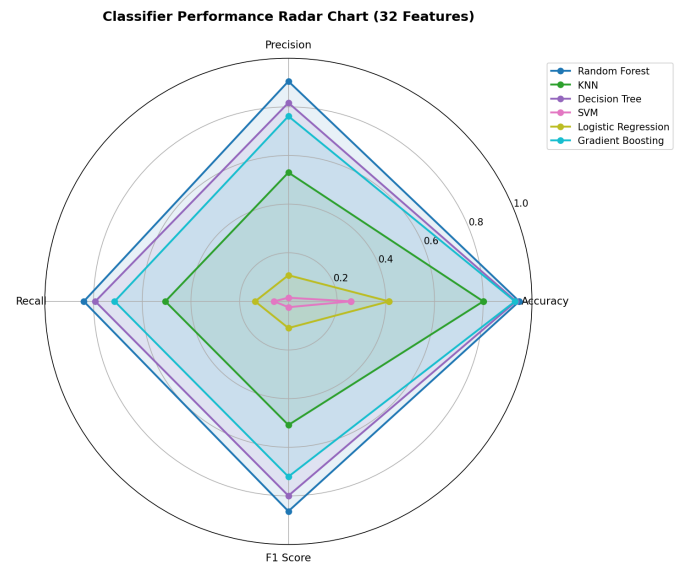


Fig. 11: Performance Radar Chart - Extended Feature Set (32 Features)

reduces extraction time from ~19 minutes (APT1) to 215–290 s on 1,150 samples, a >75% reduction. The extended (32) feature set outperforms the reduced (14) set for tree- and instance-based classifiers (RF 94.78%, DT 93.48%, KNN 80.00%); LR degrades by 10 pp under multicollinearity, and SVM is poor without feature scaling. The reduced set offers an effective speed-accuracy compromise (RF 93.91% at 215 s, vs. 290 s for extended).

Limitations. The corpus is drawn from a single source (MalwareBazaar) and may exhibit selection bias; it lacks benign samples for binary evaluation. Static header features cannot detect runtime-obfuscated or packed malware, and SVM/LR were not tuned with feature scaling or extensive grid search. The methodology is Windows PE-specific.

Future work includes expanding to 10,000+ samples with

benign files, applying feature selection (PCA, t-SNE), parallelizing extraction, exploring deep models (CNN, LSTM) for automatic feature learning [8], and evaluation in production-ready real-time detection pipelines.

ACKNOWLEDGEMENTS

We thank MalwareBazaar (abuse.ch) for providing access to malware samples.

Generative AI Disclosure: Claude (Anthropic) was used to assist with prose drafting and revision under the authors' direction. All experimental results, data, code, and figures were produced and verified by the authors.

REFERENCES

- [1] R. Kalakuntla, A. B. Vanamala, and R. R. Kolipyaka, "Cyber security," *HOLISTICA-Journal of Business and Public Administration*, vol. 10, no. 2, pp. 115–128, 2019.
- [2] P. Black and J. Opacki, "Anti-analysis trends in banking malware," in *2016 11th International Conference on Malicious and Unwanted Software (MALWARE)*, pp. 1–7, IEEE, 2016.
- [3] N. Balram, G. Hsieh, and C. McFall, "Static malware analysis using machine learning algorithms on APT1 dataset with string and PE header features," in *2019 International Conference on Computational Science and Computational Intelligence (CSCI)*, pp. 90–95, IEEE, 2019.
- [4] T. Rezaei and A. Hamze, "An efficient approach for malware detection using PE header specifications," in *2020 6th International Conference on Web Research (ICWR)*, pp. 234–239, IEEE, 2020.
- [5] R. S. Santos and E. D. Festijo, "Generating features of windows portable executable files for static analysis using portable executable reader module (pefile)," in *2021 4th International Conference of Computer and Informatics Engineering (IC2IE)*, pp. 283–288, IEEE, 2021.
- [6] N. Maleki and H. Rastegari, "An improved method for packed malware detection using PE header and section table information," *International Journal of Computer Network & Information Security*, vol. 11, no. 9, 2019.
- [7] H. H. Al-Khshali, M. Ilyas, and O. N. Ucan, "Effect of PE file header features on accuracy," in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 1115–1120, IEEE, 2020.
- [8] S. Kumar *et al.*, "MCFT-CNN: Malware classification with fine-tune convolution neural networks using traditional and transfer learning in internet of things," *Future Generation Computer Systems*, vol. 125, pp. 334–351, 2021.
- [9] C. K. Yuk and C. J. Seo, "Static analysis and machine learning-based malware detection system using PE header feature values," *International Journal of Innovative Research and Scientific Studies*, vol. 5, no. 4, pp. 281–288, 2022.
- [10] K. K. R. Choo and A. Dehghantanha, *Handbook of Big Data Analytics and Forensics*. Springer, 2022.
- [11] abuse.ch, "MalwareBazaar: Malware sample exchange," Available: <https://malwarebazaar.abuse.ch>, Accessed: 2024.